Experiment 4: Interrupt Control Using the PIC32MX370F512L Microcontroller

Objective:

The goal of this experiment is to use the interrupt capabilities of the microcontroller to create a precise time delay.

Tools/Software/Instruction manuals:

- Mplab Ide software
- PIC32MX370F512L Microcontroller
- MIPS Instruction set.
- PIC32MX370F512L datasheet.
- Basys MX3 Board reference manual.

Procedure:

A microcontroller (MCU) is a small, self-contained computing device that includes a processor (CPU), memory (RAM, Flash/EEPROM), and input/output peripherals all on a single chip. It is designed for embedded systems and real-time control applications.

The microcontroller presented in the lab looks like this,

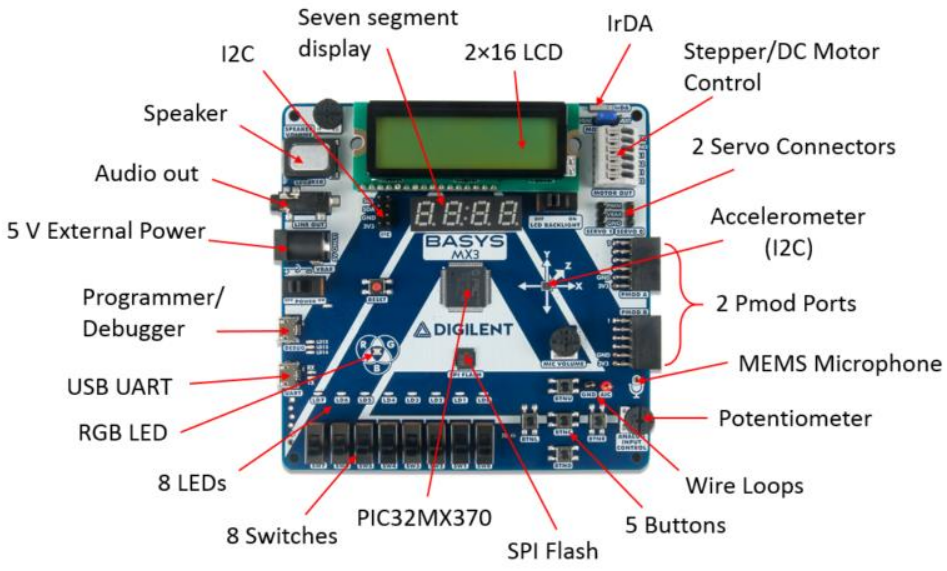

Figure 1: Basys MX3 PIC32MX Trainer Board.

(Image courtesy of Digilent, retrieved from *element14 Community*, Feb. 15, 2025).

Interrupts:

As the name indicates, an interrupt is a kind of interruption that can pause the program temporarily. When interrupt occurs, CPU pauses the execution, handles the interrupt and then resumes the flow.

The block diagram for interrupt is shown below,

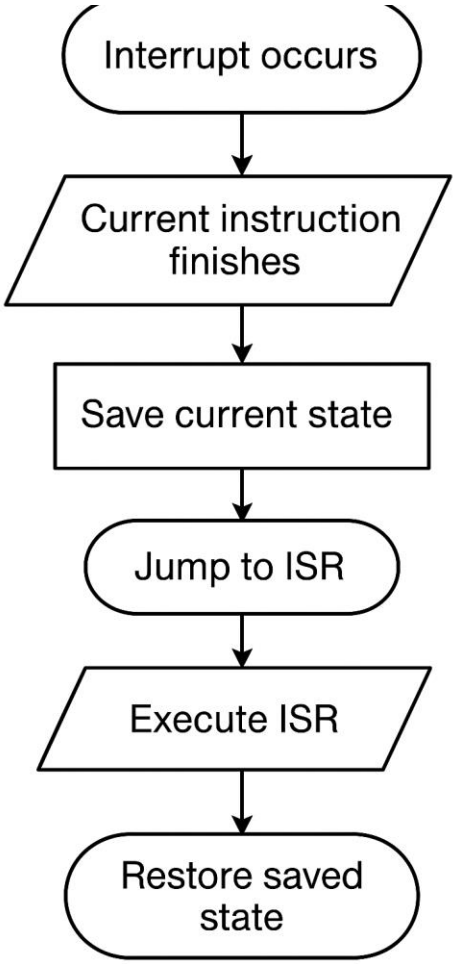

Figure 2: Interrupt Handling

As it is explained in Fig 0, when an interrupt is enabled, the main program will finish the current instruction, save its value and will not move forward until the interrupt has been cleared. To clear the interrupt, it will go to interrupt service routine. Once that has been done, now the program will move forward with the program where it left off.

For PIC32 board, if you want to utilize the interrupt, you can do so by writing to certain registers. In this program, to trigger the interrupt, a peripheral timer will be used. Timer1 is a hardware peripheral timer that is built into the microcontroller. This is a timer whose period can be controlled by the user. So, when I set the value of PR1, I'm defining the timer's period — the number of ticks before the timer rolls over. Once that value is reached, Timer1 overflows and

sets its interrupt flag. If interrupts are enabled, this causes the CPU to **pause the main program**, handle the interrupt by executing the ISR, and then **resume execution** from where it left off. Since Timer1 is a hardware unit, it automatically sets the IFS04 bit. That will tell CPU that timer1 wants an interrupt. To clear the interrupt, the user needs to clear the bit and move forward.

Timer1 is controlled by a register called T1CON. If you set up the bits of T1CON, you set up the timer 1.

The clock being used by the CPU is working at very high frequency, it will be impossible to deal with that frequency so prescaler bits are used. Prescaler will scale the clock down i.e. if your prescaler bits are set up to 11, the prescaler being used is 1:256. It means for each cycle of 10MHz, your clock is running at 10MHz/256 (approx. 39062.5 Hz). So, you have lowered the clock speed. Likewise, if your clock has a period of 10^-7s, you now have delayed the period to 2.56 * 10^(-5) s. So prescaler is used to manage the clock of your timer1 more efficiently.

Code Structure:

The code structure can be explained by following figure

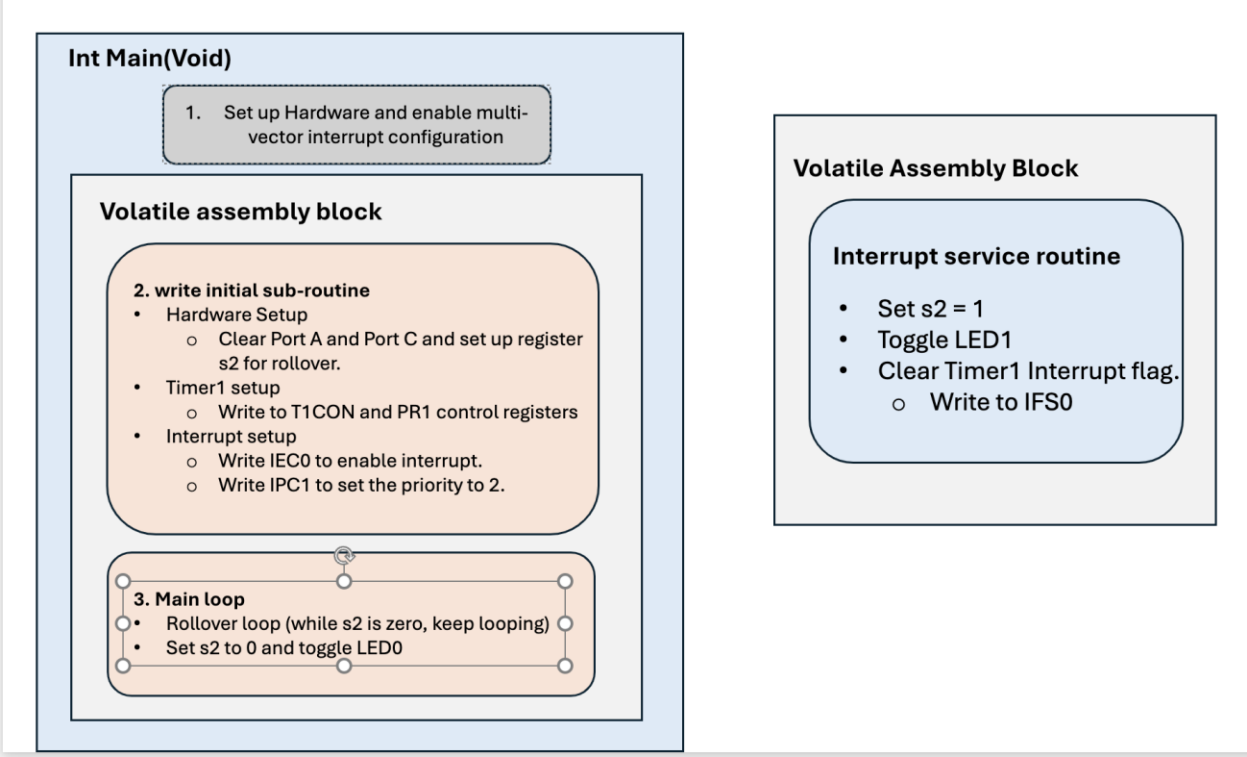

Figure 3: Code Structure

In this we can see the main structure of the code. At the start hardware is set up and multi-vector interrupt configuration is enabled using C. When this configuration is enabled, each interrupt can

have its own vector i.e. address compared to the single vector configuration in which all interrupts are being handled by the single vector.

Inside the main loop, after the C block, the initial subroutine is set up. The initial subroutine will set up hardware by loading integer 0 to s2, it will also clear any existing values at bits in porta and port C. This is done by by first setting up port a and port c as output and then they are cleared to make sure that there is nothing there.

Timer1 is controlled by T1CON and PR1. By looking at the datasheet, respective values are set at the corresponding bits for T1CON and respective value for prescaler is set at PR1. Interrupt setup is controlled by IEC0, by looking at the sheet, that can be written as well. Furthermore, the priority is set for the interrupt by using IPC1 register.

Timer1 is set up to count using a 1:256 prescaler, until it hits the value in PR1, then it triggers an interrupt. This gives you a periodic time signal.

In the ISR:

- It sets $s2 = 1, This is the rollover flag
- It toggles LED1 to show that the interrupt happened.
- It clears the interrupt flag (so future interrupts can occur)

So, the ISR is basically a "signal" that the delay time passed.

Therefore, in the Main loop, we create an inner loop that is waiting for rollover, and when that happens, it will toggle LED0 and set s2 to 0 again.

Task 1: Run the project template program on the PIC32 board and observe that LED0 turns on. Note that this LED is actually flashing at a very high frequency.

When the project template was run it was observed that LED is indeed flashing at very high frequency.

Task 2: Configure the Timer 1 control registers in order to enable use of the timer. This entails initializing registers T1CON and PR1 to the appropriate bit values. Choose the 1:256 prescaler and a timer period of 0.5 sec.

T1CON is a 32 bit register, only 16 bits are used for this assignment. Bit values for T1CON as noted from datasheet are following,

Bits	Values	Bits	values
15	To Turn on the timer, “1”	6	0
14	0/1	7	0
13	0	5-4	Prescale, 11 (for 1:256)
12	1	3	x
10	x	2	0
9	x	1	0
8	x	0	x

Table 1. T1CON configuration

The following code snippet was written,

```
/* begin TIMER1 setup */
    // configure Timer1 control register bits
        "la $s0, T1CON          \n\t"
        "li $t0, 0b1000000000110000  \n\t"  // ON=1, TCKPS=11 (1:256)
        "sw $t0, 0($s0)         \n\t"

    // Set Timer1 roll over period to MAX (0xFFFF) or PR1 = 19531
        "la $s0, PR1            \n\t"
        "li $t0, 19531  \n\t"  // PR1 = 19531
        "sw $t0, 0($s0)         \n\t"
/* end TIMER1 setup */
```

The frequency of the clock is 10MHz. Therefore, prescaler will make it, (10x10^-6)/256 = 39,062.5 Hz. This is for the period of the timer i.e. 1second. If you reduce it to 0.5seconds, frequency will also be reduced. 0.5(39,062.5 Hz) = 19531.

Therefore, by loading the address of each register (T1CON and PR1), writing an integer value and storing that value in the first register, we can successfully complete this task.

Task 3: Configure the Timer 1 interrupt control registers in order to trigger and service interrupts. This entails initializing registers IEC0 and IPC1 to the appropriate bit values. Set the timer interrupt priority level to 2.

The following values were written to IEC0 and IPC1.

```
/* begin INTERRUPT setup */
    // enable Timer1 interrupt in IEC0
            "la $s0, IEC0SET       \n\t"
            "li $t0, 0b0000000000010000  \n\t"  // Bit 4
            "sw $t0, 0($s0)        \n\t"

    // set Timer1 interrupt priority to 2 in IPC1
            "la $s0, IPC1          \n\t"
            "li $t0, 0b0000000000001000  \n\t"  // 0b010 << 2
            "sw $t0, 0($s0)        \n\t"
/* end INTERRUPT setup */
```

IPC1 holds interrupt priority/subpriority settings. For interrupt #4 (Timer1, from the datasheet), bits 4–2 are the priority, therefore you set priority = 2 (binary 010) in the correct position: bits 4–2 whereas the Subpriority remains 0 (00). Bit 4 in IEC0 is the one responsible for enabling the timer1, therefore it's set to 1.

Task 4: Write the code for the Timer 1 Interrupt Service Routine (ISR) located at the bottom of the program. This ISR is responsible for acknowledging and servicing the interrupt from the timer rollover. Inside this ISR, write the code to clear the Timer 1 interrupt flag bit in the IFS0 register. This will ensure that the interrupt can be triggered repeatedly.

Following code was written or this task,

```
// clear TIMER1 interrupt FLAG
    "la $s0, IFS0CLR  \n\t"
    "li $t0, 0b0000000000010000  \n\t"
    "sw $t0, 0($s0)  \n\t"

/* end ISR code */
```

To clear the interrupt flag, IFS0 is used. By simply loading the address of IFS0CLR and writing integer value for the bit we are supposed to clear, this task was achieved.

In this case the bit is 4 as mentioned in the previous task.

Task 5: If all the previous steps have been coded correctly up to this point, then LED1 should be flashing every 0.5 sec. At this point, the Timer 1 interrupt is set up and ready to be used to create a delay.

The LED was indeed flashing every 0.5 seconds. (See video attached to the email)

Task 6: Inside the main loop in the program, write the code for a loop that breaks out every time the interrupt trigger occurs, which signals that the timer has rolled over. This loop should be created using conditional statements in order to loop indefinitely until the interrupt is triggered by Timer 1. If this is done correctly, LED0 and LED1 will blink every 0.5 sec and there will be a precise pulse train, with a 50% duty cycle and a 1 sec period, being generated at pin JA7 of PMODA.

The code for this task is as follows,

```
/* ************************************************************************** */
    /* begin MAIN loop */
         "MainLoop:  \n\t"

           // inner loop waiting for rollover
        "Rollover:     \n\t"
           "beq $s2, $zero, Rollover \n\t"

           // set rollover parameter in register to value 0
           "li $s2, 0  \n\t"

           // toggle pin LED0 to test delay functionality by inspection
           "la $s0, PORTAINV   \n\t"
           "li $t0, 0b0000000000000001  \n\t"
           "sw  $t0, 0($s0) \n\t"

           // toggle pin RC3 of PORTC to measure delay using oscilloscope
           "la $s0, PORTCINV   \n\t"
           "li $t0, 0b0000000000001000  \n\t"
           "sw  $t0, 0($s0) \n\t"

        "j MainLoop \n\t"
    /* end MAIN loop  */
```

The rollover loop in the main loop keeps going as long as s2 is equal to 0. But when the interrupt happens, and s2 is set to 1, the loop breaks. The register s2 is set again to 0. PORTAINV and PORTCINV are special registers that are used to toggle the states. If you write 1, the toggling will happen and if you write 0, nothing will happen. Therefore, those are used to toggle the values at PORTA bit 0 and pin RC3 of PORTC

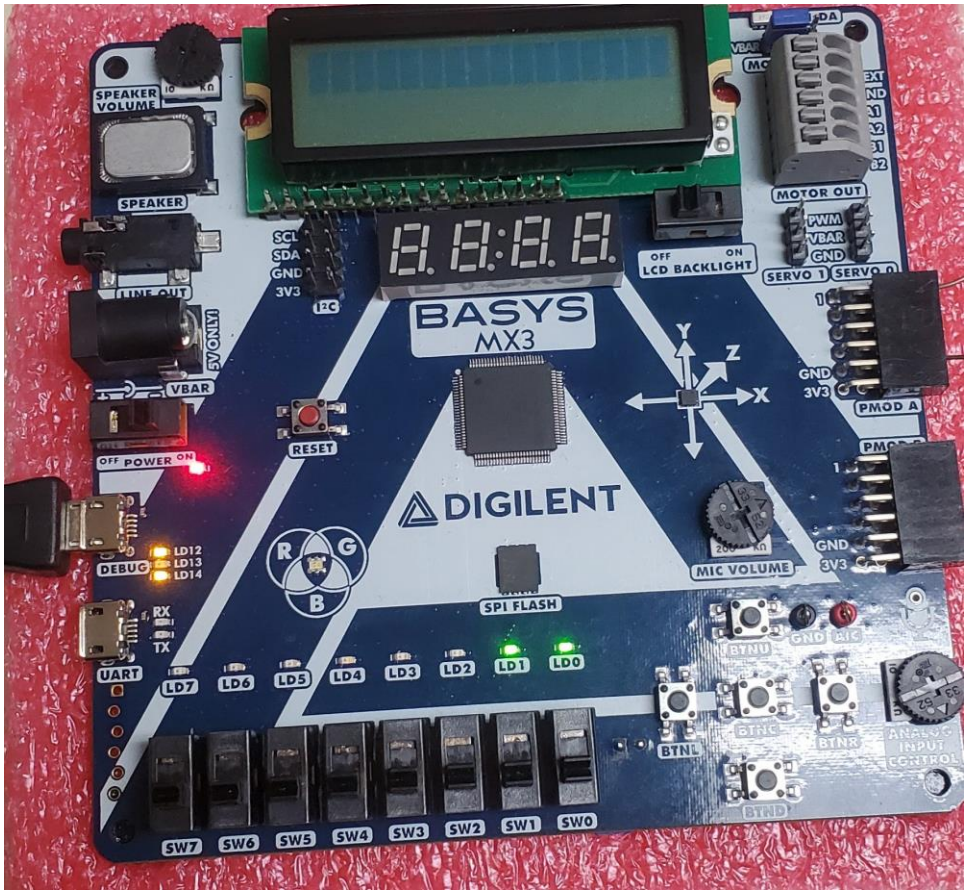

Fig 5. Shows Both LED0 and LED1 toggling

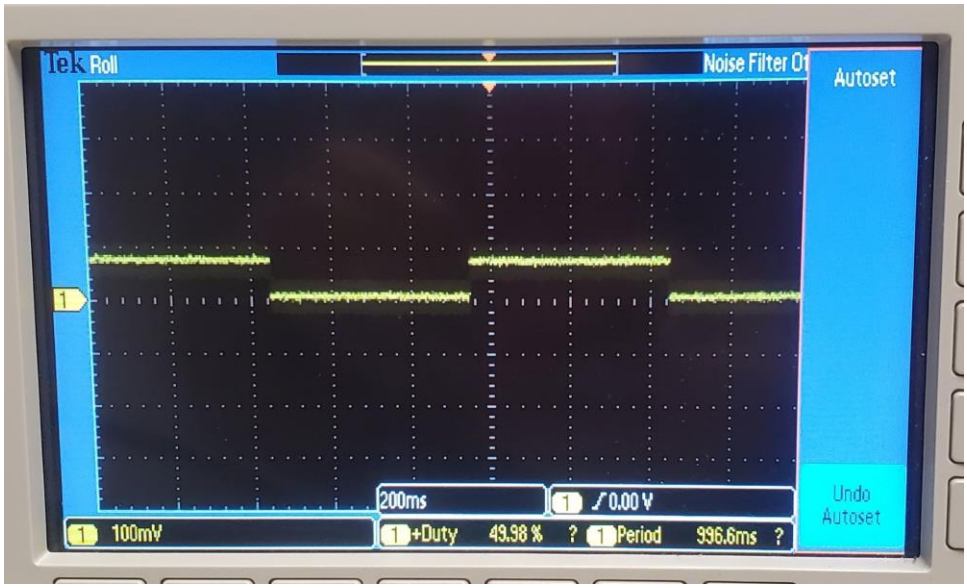

Fig 6. Oscilloscope trace of 50% duty cycle and approx. 1s period

Task 7: Set the timer period to the maximum value by changing the appropriate bits in register PR1 and run your code again. Use the oscilloscope to measure the pulse train to verify that his maximum period is achieved.

The maximum value can be set by realizing that it has to be all 1's for the 16 bits, that turns out to be 0xFFFF in hexadecimal values. This is 1111 1111 1111 in binary. In decimal that number is 65535. It represents the maximum value for 16-bit integers. Once we put that value following trace is observed,

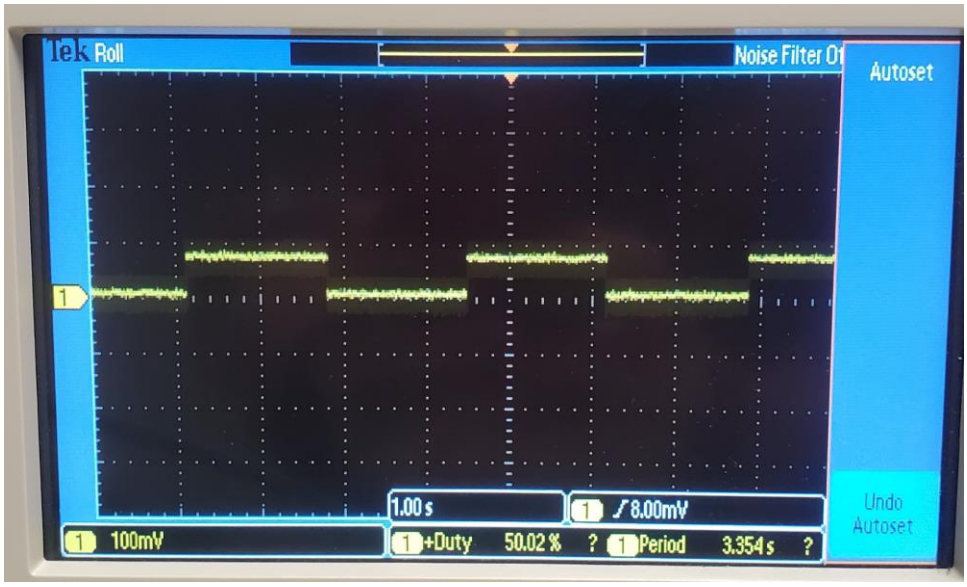

Figure 6: Maximum period observed

Notice how the period of the trace is 3.354s. This is not the period of the original clock. The period of CPU clock is 1/10MHz that is 10^-7s. The period of the timer1 is 256*10^-7s. However, the period being observed is 3.354s. The period observed is calculated,

$$T_{\text{timer clock}} = \frac{10\,\text{MHz}}{256} = 39.0625\,\text{kHz}$$

$$T_{\text{tick}} = \frac{1}{39.0625\,\text{kHz}} = 25.6\,\mu\text{s}$$

$$T_{\text{overflow}} = 65535 \times 25.6\,\mu\text{s} = 1677.77\,\text{ms} \approx 1.68\,\text{seconds}$$

The observed time period should be 1.68 seconds, however the observed value at oscilloscope is 3.354s. Since PMODA JA7 and LED0 are toggling at the same time in the main loop, and both

toggles are visible on the oscilloscope, we're observing both signals toggling simultaneously. The 3.354s period is the combined effect of the toggles happening together, and since both signals are toggling at the same time, the oscilloscope shows us twice the expected period.